Stage 1: Speech-to-Text with Whisper

Theory notes and interview questions
Project: Doctor–Patient Conversation Audio -> SOAP Notes

What this stage does: takes an audio file and produces (1) the full transcript, (2) segment-level timestamps and (3) word-level timestamps, saved as JSON for later stages.

Contents

1. Where STT fits in the pipeline
2. Audio fundamentals
3. From sound to a log-Mel spectrogram
4. Whisper's architecture
5. How Whisper was trained
6. Decoding: turning probabilities into text
7. Handling audio longer than 30 seconds
8. How timestamps are produced
9. faster-whisper vs openai-whisper
10. Our code, line by line
11. The JSON output
12. Limitations in a clinical setting
13. Measuring quality: WER
14. Interview questions and answers

1. Where STT fits in the pipeline

```
Audio file
   |
   v
[1] Speech-to-Text (Whisper)      <-- THIS STAGE: what was said, and when
   v
[2] Speaker diarization           who spoke when (Speaker A / B)
   v
[3] Timestamp alignment           attach a speaker to every word
   v
[4] Role identification           Speaker A = doctor, B = patient
   v
[5] Clinical NER (BioBERT)        symptoms, drugs, doses, durations
   v
[6] SOAP note generation          Subjective / Objective / Assessment / Plan
```

STT is the foundation: every later stage works on its output. Any word Whisper gets wrong (e.g. a drug name) becomes an error in the final SOAP note, so STT quality caps the quality of the whole system. STT knows *what* and *when*, but not *who* - that is diarization's job.

2. Audio fundamentals

- **Sound** is air pressure changing over time. A microphone turns it into a voltage; the computer measures that voltage many times per second.
- **Sample rate** = measurements per second. CD audio is 44,100 Hz. Whisper uses **16,000 Hz**: enough for speech (human speech energy is mostly below 8 kHz; by the Nyquist theorem you need 2x the highest frequency), and smaller to process.
- **Channels**: stereo has 2; Whisper uses **mono** (1). Stereo is averaged down.
- **Decoding**: mp3 / m4a / wav are containers + compression formats. Before Whisper sees anything, the file is decoded to raw numbers (a float array between -1 and 1). In our project the **PyAV** library (the *av* package) does this, so ffmpeg is not needed as a separate install.
- Our 14.21 s test clip at 16 kHz = 14.21 x 16,000 = about **227,000 samples** (we saw 227,426 when we tested decode_audio).

3. From sound to a log-Mel spectrogram

Neural networks do not work well on raw samples directly, so Whisper first converts audio into a **spectrogram**: a picture with time on the x-axis, frequency on the y-axis and loudness as brightness.

- **Short windows**: the audio is cut into 25 ms windows (400 samples), moving forward 10 ms (160 samples) each step. So there are **100 frames per second**.
- **FFT** (Fast Fourier Transform): for each window, compute how much energy is at each frequency.
- **Mel scale**: humans hear pitch roughly logarithmically (100->200 Hz sounds like a bigger jump than 5000->5100 Hz). Frequencies are grouped into **80 Mel bins** (128 for large-v3) that mimic the ear.
- **Log**: loudness is also perceived logarithmically, so values are log-scaled and normalised.
- Result for one 30-second window: an **80 x 3000** matrix (80 Mel bins x 3000 frames). Shorter audio is padded with silence up to 30 s.

4. Whisper's architecture

Whisper is an **encoder-decoder Transformer** (sequence-to-sequence), the same family as machine translation models: it 'translates' audio into text.

```
log-Mel (80 x 3000)
   -> 2 x Conv1D (2nd has stride 2)  -> 1500 frames   (each = 20 ms of audio)
   -> + positional encoding
   -> ENCODER: N Transformer blocks (self-attention)  -> 1500 audio vectors
                                                          |
   DECODER: N Transformer blocks                          |
     self-attention over tokens written so far            |
     CROSS-ATTENTION to the audio vectors  <--------------+
   -> probability of every token in the vocabulary  -> pick next token, repeat
```

- **Encoder**: reads the whole spectrogram at once and produces 1500 vectors - one per 20 ms of audio - each describing what is happening at that moment in context.
- **Decoder**: writes text one **token** at a time (autoregressive). At each step it looks at the tokens already written (self-attention) and at the audio (cross-attention).
- **Tokens**: pieces of words from a byte-level BPE tokenizer (~51,865 tokens for multilingual models). Rare words like 'paracetamol' are split into several sub-word tokens.
- **Special tokens** control behaviour, e.g. <|startoftranscript|> <|en|> <|transcribe|> and timestamp tokens <|0.00|> ... <|30.00|>.

Model	Parameters	Download (approx.)	Notes
tiny	39 M	75 MB	very fast, many errors
base	74 M	145 MB	quick tests
small	244 M	480 MB	**our default**: good CPU trade-off
medium	769 M	1.5 GB	noticeably better on hard audio
large-v3	1550 M	3 GB	best accuracy, slow on CPU, 128 Mel bins

5. How Whisper was trained

- About **680,000 hours** of audio paired with transcripts collected from the internet (large-v3: about 5 million hours including pseudo-labelled data).
- **Weakly supervised**: the transcripts were not hand-checked by experts, but the sheer amount makes the model robust to accents, noise and recording conditions.
- **Multitask**: one model learned transcription, translation to English, language identification, voice-activity (no-speech) detection and timestamp prediction - selected by special tokens.
- Because transcripts came from subtitles and human-written text, Whisper learned to output **punctuation, capitals and normalised numbers** ('five hundred' -> '500'). It writes 'readable text', not a verbatim record of every 'um'.

6. Decoding: turning probabilities into text

At each step the decoder gives a probability for every token. How we choose tokens is **decoding**.

- **Greedy**: always take the most likely token. Fast, but one early mistake cannot be undone.
- **Beam search** (we use beam_size=5): keep the 5 best partial sentences at each step and choose the best complete one. Slower, usually more accurate.
- **Temperature fallback**: if the result looks broken - too repetitive (gzip **compression ratio** > 2.4) or low confidence (average log-probability < -1.0) - Whisper retries with more randomness (temperature 0.2, 0.4, ... 1.0).

- **No-speech detection**: Whisper predicts a 'no speech' probability; if high (> 0.6) and the text is low-confidence, the window is treated as silence.
- **Language detection**: if no language is given, Whisper looks at the first 30 s and predicts a language token. Our run: 'en' with probability 0.99.

7. Handling audio longer than 30 seconds

Whisper's encoder only accepts exactly 30 seconds. Long audio is processed with a **sliding window**:

- Transcribe window 1 (0-30 s).
- Find the last complete timestamp token, e.g. <|27.40|>. Move the window start (the 'seek' position) to 27.40 s so no word is cut in half.
- Transcribe the next 30 s. By default the previous text is given to the decoder as a **prompt** (condition_on_previous_text) so spelling and style stay consistent.
- Downside: errors and hallucinations can carry over between windows. This is also why our 'live' output arrives in batches - segments are produced one 30 s window at a time.

8. How timestamps are produced

8.1 Segment timestamps - predicted directly

Whisper has 1501 timestamp tokens: <|0.00|>, <|0.02|> ... <|30.00|> (0.02 s steps, matching the 20 ms encoder frames). The decoder writes them just like words:

```
<|0.00|> Good morning, what brings you in today?<|2.08|>
<|2.84|> I have been experiencing fever ...<|6.42|>
```

Text between a start and end token is one **segment**. This is why our times are multiples of 0.02 (2.08, 2.84, 6.42). Segments follow pauses and sentence structure, **not speakers**.

8.2 Word timestamps - inferred afterwards (cross-attention + DTW)

- While writing each token, the decoder's **cross-attention** shows which of the 1500 audio frames it focused on. Certain attention heads ('alignment heads') are known to track time well.
- Collect these weights into a matrix: **text tokens x audio frames**.
- **Dynamic Time Warping (DTW)** finds the best monotonic path through the matrix: tokens must be in order and time must only move forward. Each token gets a range of frames.
- Sub-word tokens are merged into words; frame index x 0.02 = seconds.
- Accuracy: roughly **±0.1-0.2 s**. Good enough to assign speakers, not phoneme-precise. A later 'forced alignment' step (e.g. wav2vec2 in WhisperX) can refine them.

probability in our JSON is the model's confidence in the word's *text* (average token probability), not in its timing.

9. faster-whisper vs openai-whisper

	openai-whisper	faster-whisper (ours)
Model weights	OpenAI Whisper	Same weights, converted
Inference engine	PyTorch	CTranslate2 (optimised C++)
Speed on CPU	1x	about 4x faster, less memory

	openai-whisper	faster-whisper (ours)
Audio decoding	needs ffmpeg installed	PyAV, bundled
Quantization	fp16 / fp32	int8, int8_float16, fp16 ...
VAD filter	no	optional Silero VAD (vad_filter=True)
Install size	~1 GB with PyTorch	~150 MB

Quantization (int8): model weights are stored as 8-bit integers instead of 32-bit floats - about 4x smaller and faster on CPU, with a very small accuracy loss.

VAD (Voice Activity Detection): a small model that finds speech regions and skips silence. Helps prevent hallucinations on long silent parts. We left it off so you see Whisper's raw behaviour.

10. Our code, line by line

Step in transcribe.py	What happens
WhisperModel('small', device='cpu', compute_type='int8')	Downloads (first time) and loads the model into memory, quantized to int8.
model.transcribe(audio, word_timestamps=True, beam_size=5)	Decodes audio with PyAV, detects language, returns **info** and a **lazy generator** of segments.
for seg in segments:	Each iteration actually runs Whisper on the next part of the audio - this is where the work happens, and why we can print results live.
seg.words	Word list with start, end, probability (only because word_timestamps=True).
' '.join(segment texts)	Builds full_transcript.
json.dumps(...)	Saves outputs/<name>.json for the next stages.

Requirements: **faster-whisper==1.2.1** and **av==18.1.0**. av was pinned because av 19 removed the *metadata_errors* argument that faster-whisper 1.2.1 still passes - a real-world example of why dependency versions are pinned.

11. The JSON output

```
{
  "audio_file": "audio/sample_consultation.wav",
  "model": "small",  "language": "en",  "duration": 14.21,
  "full_transcript": "Good morning, what brings you in today? I have been ...",
  "segments": [ {"id": 1, "start": 0.0, "end": 2.08,
                 "text": "Good morning, what brings you in today?"}, ... ],
  "words":    [ {"word": "Good", "start": 0.0, "end": 0.2,
                 "probability": 0.92, "segment_id": 1}, ... ]
}
```

- **Why keep words separately?** Diarization gives speaker time ranges; a single segment can contain two speakers, so speaker assignment is done per word (e.g. by the word's midpoint).
- **Why keep probability?** Low-confidence words (our 'milligrams' p=0.38, 'three' p=0.50) flag clinically important terms for review.

- **Gaps**: segment 1 ends 2.08, segment 2 starts 2.84 - the 0.5 s silence we inserted between speakers.

12. Limitations in a clinical setting

- **Hallucination**: on silence, music or noise Whisper may invent fluent text (e.g. 'Thank you for watching'), because it learned from subtitles. Mitigations: VAD, no-speech threshold, condition_on_previous_text=False.
- **Medical vocabulary**: drug names, dosages and abbreviations are rare in training data. Mitigations: larger model, *initial_prompt* with expected terms, fine-tuning on medical audio, post-correction with a drug dictionary.
- **Normalisation**: '500' vs 'five hundred', filler words removed - text is cleaner than what was said.
- **Overlapping speech**: when doctor and patient talk at once, Whisper usually transcribes only one.
- **No speaker information** - solved in stage 2.
- **Privacy**: patient audio is protected health information (HIPAA / GDPR). Running Whisper locally, as we do, keeps audio on the machine - a strong reason to prefer local models over cloud APIs.

13. Measuring quality: Word Error Rate (WER)

WER = (S + D + I) / N where S = substituted words, D = deleted words, I = inserted words and N = number of words in the correct (reference) transcript. Lower is better; it can exceed 100% if there are many insertions.

```
Reference : take paracetamol twice a day      (N = 5)
Hypothesis: take paracetamol three times day
  twice -> three  (S), 'times' inserted (I), 'a' deleted (D)
  WER = (1 + 1 + 1) / 5 = 60%
```

Text is usually normalised first (lower-case, remove punctuation, '500' = 'five hundred'). In medicine, WER alone is misleading: one wrong drug name is worse than ten wrong 'the's - so teams also measure **medical-term error rate**.

14. Interview questions and answers

Q1. What is Whisper?

An open-source speech recognition model from OpenAI (2022). It is an encoder-decoder Transformer trained on about 680k hours of weakly supervised multilingual audio. It does transcription, translation to English, language identification and timestamp prediction in one model.

Q2. Why did you choose Whisper for a medical transcription project?

It is robust to accents and noise, produces punctuation and timestamps, is open source and runs locally - important for patient privacy. Its accuracy is close to commercial APIs without sending audio to a third party.

Q3. Why faster-whisper instead of the original package?

Same model weights, but run on CTranslate2: about 4x faster on CPU, lower memory with int8 quantization, no PyTorch or ffmpeg dependency, and reliable word timestamps. It is also the base of WhisperX, useful for later alignment.

Q4. What is the input to the Whisper model?

A log-Mel spectrogram of 30 seconds of 16 kHz mono audio: 80 Mel bins x 3000 frames (25 ms windows, 10 ms hop). Shorter audio is padded, longer audio is processed in sliding 30 s windows.

Q5. What is a Mel spectrogram and why use it?

A time-frequency picture of audio where frequencies are grouped on the Mel scale, which matches human pitch perception, and loudness is log-scaled. It is compact and emphasises information relevant to speech.

Q6. Explain the encoder and decoder roles.

The encoder converts the spectrogram into 1500 contextual audio vectors (20 ms each). The decoder generates text tokens autoregressively, using self-attention over previous tokens and cross-attention over the audio.

Q7. How does Whisper produce timestamps?

Segment timestamps are special tokens (0.02 s resolution) predicted by the decoder. Word timestamps are inferred by taking cross-attention weights from alignment heads and running Dynamic Time Warping to map each token monotonically onto audio frames.

Q8. How accurate are Whisper's word timestamps?

Roughly ±0.1-0.2 s. Fine for speaker assignment; for precise alignment use forced alignment with a phoneme model such as wav2vec2 (as WhisperX does).

Q9. What is Dynamic Time Warping?

An algorithm that finds the lowest-cost alignment between two sequences of different speeds under the constraint that both move forward in order. Here: text tokens vs audio frames.

Q10. What is beam search? Why beam_size=5?

It keeps the k most probable partial transcripts at each step instead of only the best one, so an early mistake can be recovered. 5 is Whisper's recommended trade-off between accuracy and speed.

Q11. What are Whisper hallucinations and how do you reduce them?

Fluent text invented on silence or noise, a side-effect of training on subtitles. Reduce with VAD filtering, no-speech threshold, compression-ratio / log-prob thresholds with temperature fallback, and disabling condition_on_previous_text.

Q12. How does Whisper handle long recordings?

Sliding 30 s windows. After each window it seeks to the last predicted timestamp and continues, optionally prompting with the previous text for consistency.

Q13. What is quantization and what did int8 change?

Storing weights with fewer bits. int8 makes the model about 4x smaller than fp32 and faster on CPU, with a minor accuracy loss.

Q14. What is WER? Compute it for an example.

WER = (S + D + I) / N. Reference 'take two tablets daily' vs hypothesis 'take to tablets' = 1 substitution + 1 deletion over 4 words = 50%.

Q15. Why is WER not enough for clinical transcription?

All errors count equally, but a wrong drug or dose is far more dangerous than a wrong article. Use a medical-term or entity-level error rate and human review of low-confidence words.

Q16. How would you improve accuracy on medical terms?

Larger model, initial_prompt with expected vocabulary, fine-tuning on medical speech (e.g. with LoRA), post-correction against a drug lexicon, and flagging low-probability words for review.

Q17. Does Whisper know who is speaking?

No. It outputs text and time only. Speaker diarization (e.g. pyannote) is a separate model; we then merge the two using timestamps.

Q18. Why save word-level timestamps if you have segments?

A segment can span two speakers. Assigning speakers per word (via word midpoint vs diarization ranges) handles turn changes inside a segment.

Q19. What does the probability field mean?

The model's confidence in the word's text (average of its token probabilities). It does not measure timestamp accuracy.

Q20. Why is the segments variable a generator, and why does it matter?

faster-whisper transcribes lazily: work happens as you iterate. You can stream results as they are produced, but you must consume the generator to actually run transcription.

Q21. Why 16 kHz?

Speech information is mostly below 8 kHz; by Nyquist, 16 kHz sampling captures it. Lower rates mean less compute. Whisper resamples any input to 16 kHz.

Q22. What privacy concerns exist and how does your design address them?

Patient audio is protected health information under HIPAA/GDPR. Running Whisper locally avoids sending audio to third-party APIs; outputs should be stored securely and access-controlled.

Q23. What would you do if transcription is too slow?

Smaller model, int8 quantization, GPU (device='cuda'), VAD to skip silence, batched inference (BatchedInferencePipeline), or distilled models such as distil-whisper.

Q24. Name an issue you hit while building this stage.

PyAV 19 removed the metadata_errors argument used by faster-whisper 1.2.1, causing a TypeError. Fixed by pinning av==18.1.0 - showing why dependencies should be pinned.